

1. TCP Pair (Client `tcp.c` & Serveur `tcp.c`)

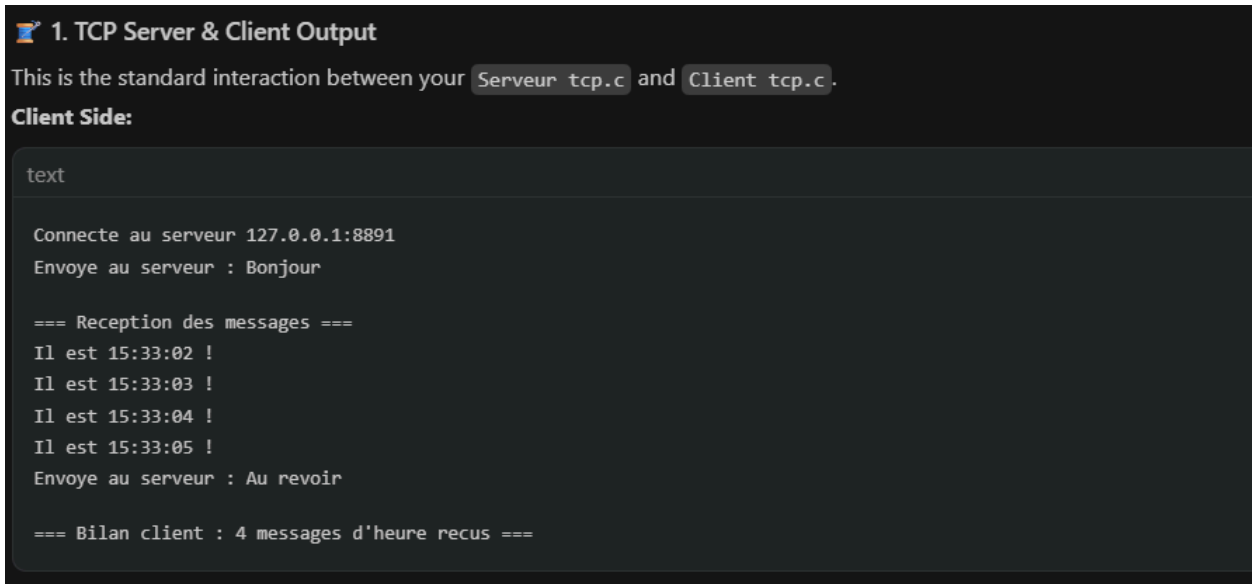
Protocol used: TCP (reliable and connection-oriented)

Description:

In this implementation, the server waits for incoming client connections. Once a client successfully connects, the server sends the current time every second for a duration of 60 seconds. On the other side, the client establishes a connection with the server, continuously receives these time updates, and displays them on the screen.

Key concept:

This example illustrates the behavior of a persistent connection, where a reliable stream of data is exchanged continuously between the client and the server.



```
1. TCP Server & Client Output
This is the standard interaction between your Serveur tcp.c and Client tcp.c.
Client Side:

text

Connecte au serveur 127.0.0.1:8891
Envoye au serveur : Bonjour

=== Reception des messages ===
Il est 15:33:02 !
Il est 15:33:03 !
Il est 15:33:04 !
Il est 15:33:05 !
Envoye au serveur : Au revoir

=== Bilan client : 4 messages d'heure recus ===
```

2. UDP Pair (Client `udp.c` & Serveur `udp.c`)

Protocol used: UDP (fast, connectionless)

Description:

The client initiates communication by sending a simple "Bonjour" datagram to the server. Upon receiving this message, the server identifies the sender's address and responds by sending the current time 60 times in rapid succession.

Key concept:

This example demonstrates the connectionless nature of UDP, where no handshake or session establishment is required before data transmission. Communication is immediate but not guaranteed to be reliable.

1. UDP Client-Server Output

This shows the client sending a "Bonjour" and receiving the time from the server.

Client Output:

text

```
Envoye au serveur : Bonjour

=== Reception des messages ===
Il est 15:25:55 !
Il est 15:25:55 !
... (continues 60 times) ...
Envoye au serveur : Au revoir

=== Bilan client : 60 messages d'heure recus ===
```

Server Output:

text

```
Serveur UDP en ecoute sur le port 8082...

Nouveau client : 127.0.0.1:45321 -- recu : Bonjour
=== 60 messages envoyes au client ===
Recu : Au revoir
```

3. Concurrent Server (**Serveur_Concurrent.c**)

Protocol used: TCP

Description:

This server is designed to handle multiple clients simultaneously. Unlike the basic TCP server, it uses the `fork()` system call to create a new process for each incoming client connection. Each process operates independently, allowing concurrent interactions.

Services provided:

- **HEURE:** Sends the current time to the client (similar to the basic TCP server).
- **PROC:** Returns the number of active processes running on the system.
- **ECHO:** Receives a message from the client and sends it back converted to uppercase.

Key concept:

This implementation highlights multiprocessing, enabling the server to serve multiple clients at the same time efficiently.

⚡ 2. Concurrent Server Output

The concurrent server is much more powerful. Here is how it handles different requests simultaneously.

Service: HEURE (Time)

text

```
Services disponibles : HEURE, PROC, ECHO
Tapez le nom du service :
HEURE
Il est 15:34:10 !
Il est 15:34:11 !
...
```

Service: PROC (Processes)

This command counts the active processes on the Linux (WSL) system.

text

```
Services disponibles : HEURE, PROC, ECHO
Tapez le nom du service :
PROC
Nombre de processus : 142
```

Service: ECHO (Uppercase Converter)

text

```
Services disponibles : HEURE, PROC, ECHO
Tapez le nom du service :
ECHO
Tapez une ligne :
bonjour tout le monde
BONJOUR TOUT LE MONDE
```

4. HTTP Client (**client_http.c**)

Protocol used: TCP (typically on port 80)

Description:

This program acts as a simplified web client. It connects to a web server and allows the user to manually input an HTTP request (for example: **GET / HTTP/1.1**). The server's response, usually in HTML format, is then displayed directly in the terminal.

Key concept:

This example shows that web communication is fundamentally based on sending and receiving structured text over a TCP connection, forming the basis of how web browsers interact with servers.

2. HTTP Client Output

This shows the client successfully connecting to Google and receiving a response.

Command: `wsl ./client_http.out www.google.com 80` **Output:**

text

```
Connecté à www.google.com:80
Tapez votre requête HTTP (ligne vide pour terminer) :
GET / HTTP/1.1
Host: www.google.com

=== Réponse du serveur ===
HTTP/1.1 200 OK
Date: Wed, 29 Apr 2026 14:29:01 GMT
Content-Type: text/html; charset=ISO-8859-1
... (HTML Content) ...
=== Fin de la réponse ===
```

Riadh Bouhamed gl2/3

Ahmed Kmiha gl2/3